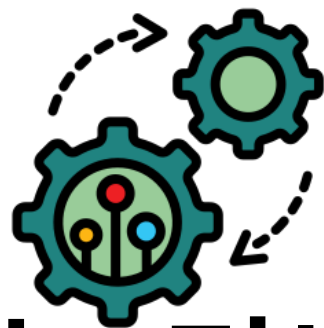




케이녹 리버싱 2회차 과제

17기 최동현 (202511688)

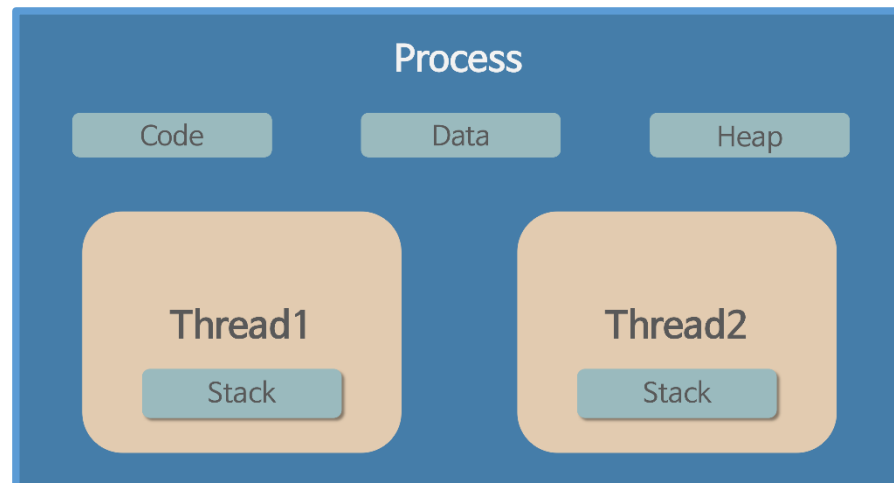
1-1. OS Architecture (ps, thread)

- 프로세스 (Process)

： 디스크에 저장된 실행 파일(PE/ELF)이 메모리에 적재되어 운영체제로부터 고유한 **가상 주소 공간(Virtual Address Space)**을 할당받아 실행 중인 상태입니다. 각 프로세스는 독립적인 **코드(Code)**, **데이터(Data)**, **힙(Heap)** 영역을 가지며, 다른 프로세스의 메모리 영역에 직접 접근할 수 없습니다(메모리 격리)

- 스레드 (Thread)

： 프로세스 내부에서 CPU 자원을 할당받아 명령어를 실행하는 실제 실행 흐름의 최소 단위입니다. 하나의 프로세스 내에 여러 스레드가 존재할 수 있으며, 이들은 프로세스의 코드, 데이터, 힙 영역을 공유하지만, 함수 호출을 관리하기 위한 **스택(Stack)**과 **CPU 레지스터 컨텍스트(Register Context)**는 스레드별로 독립적으로 할당받습니다.



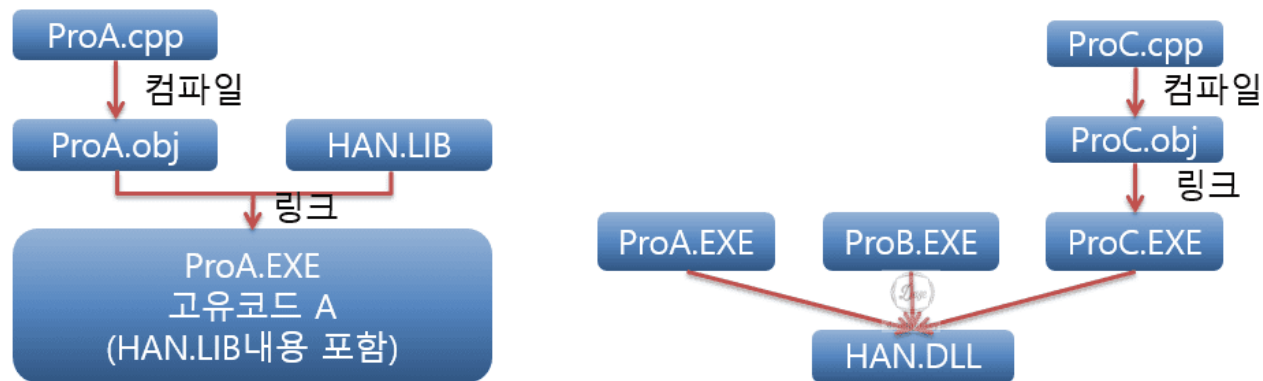
1-2. OS Architecture (module, handle)

• 모듈 (Module)

： 실행 중인 프로세스의 가상 메모리 공간에 로드된 독립적인 실행 파일(.exe) 또는 동적 링크 라이브러리(.dll, .so)를 의미합니다. 프로세스는 여러 모듈이 메모리에 맵핑(Mapping)되어 상호작용하는 구조입니다.

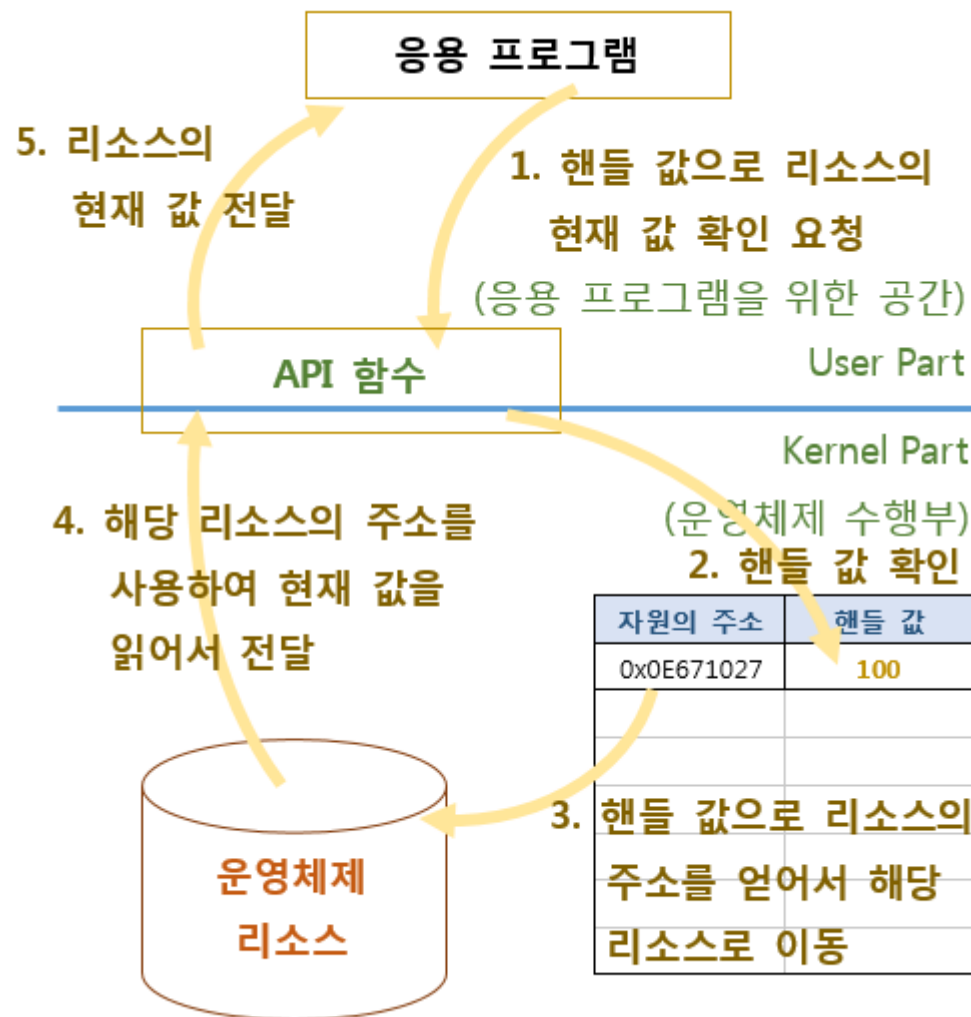
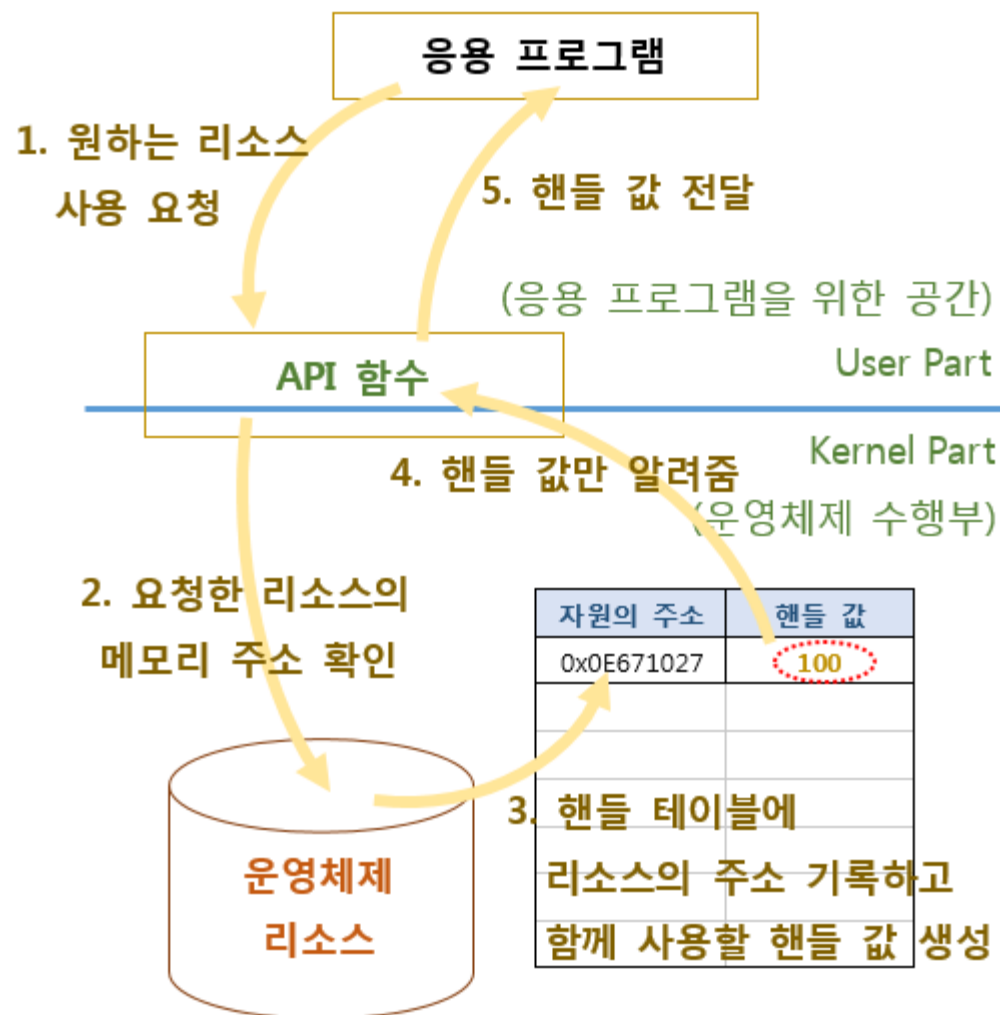
• 핸들 (Handle)

： 운영체제 커널(Kernel)이 관리하는 리소스 객체(프로세스, 스레드, 파일, 소켓 등)를 유저 모드(User Mode) 애플리케이션이 참조하고 제어할 수 있도록 부여하는 추상화된 정수형 식별자(Integer Index)입니다. 핸들을 통해 커널 내부의 실제 객체 주소에 간접적으로 접근하게 됩니다.



1-2. OS Architecture

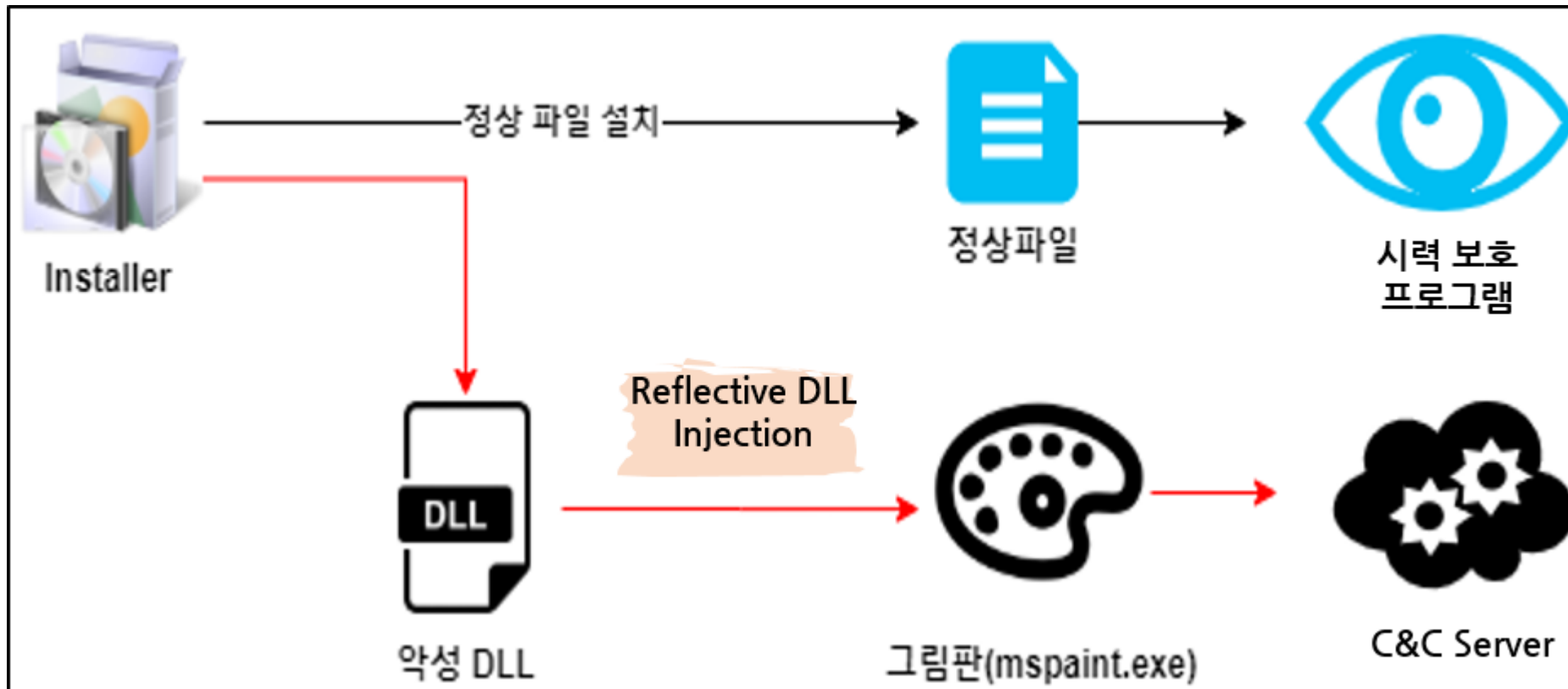
<https://blog.naver.com/tipsware/221065382244>



1-3. OS Architecture (보안 관점)

• DLL Injection & Handle Hijacking

： 악성코드 및 게임 해킹은 타겟 프로세스의 메모리 공간에 침투하기 위해 높은 권한의 핸들을 획득합니다. 획득한 핸들을 이용해 타겟 프로세스 내부에 악성 DLL의 경로 문자열을 기록하고, 타겟 프로세스 내부에 새로운 스레드를 강제로 생성시켜 악성 모듈(DLL)을 로드 시킵니다. 이 과정을 DLL Injection 이라고 합니다.



2-1. Debugger & Disassembler

- 디버거 (Debugger)

： 실행 중인 프로세스에 결합(Attach)하여 레지스터, 메모리, 스택의 상태를 동적으로 확인하고 명령어의 실행 흐름을 제어(Step, Run)하는 **동적 분석(Dynamic Analysis)** 도구입니다.

- 중단점 (Breakpoint)

： 디버거가 실행 중인 프로세스를 특정 메모리 주소에서 강제로 일시 정지시키는 메커니즘입니다.

- **소프트웨어 중단점:** 디버거가 타겟 메모리의 명령어 1바이트를 시스템 인터럽트 명령어인 INT 3 (0xCC)로 덮어씁니다. CPU가 이 명령어를 실행하면 SIGTRAP 예외가 발생하고, OS가 디버거에게 제어권을 넘깁니다. 디버거는 정지 후 원래 기계어로 다시 복원합니다. (0xCC : 일시 중지 OPCODE)
- **하드웨어 중단점:** CPU 내부의 디버그 레지스터(DR0~DR3)에 특정 주소를 저장하여 메모리 접근 시 예외를 발생시킵니다. 코드를 변조하지 않습니다.
(CPU 안 "이 주소 지켜봐!" 라는 감시표 등록, 프로그램이 그 주소의 코드를 실행하면 CPU가 바로 멈추고 디버거한테 알려줌)

2-2. Debugger & Disassembler

• 디스어셈블러 (Disassembler)

： 바이너리 파일 내부의 기계어(Machine Code, Opcode)를 파싱하여 사람이 읽을 수 있는 어셈블리어(Assembly) 텍스트로 1:1 매칭(변환)해 주는 **정적 분석(Static Analysis)** 도구입니다.

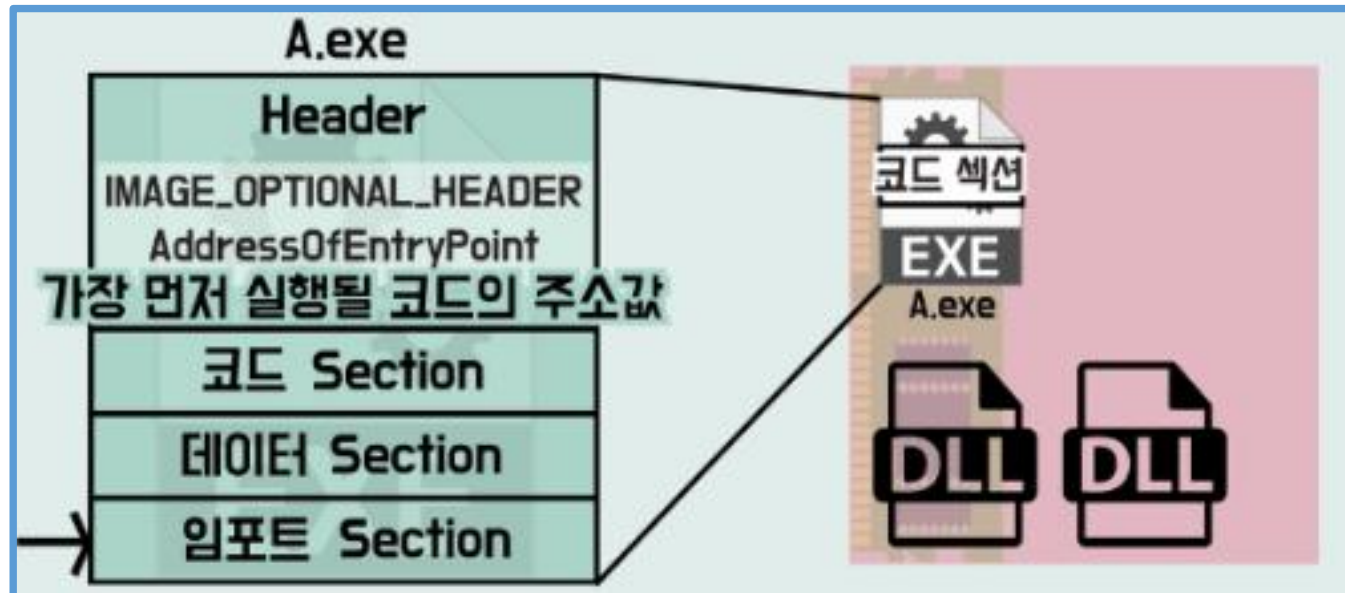
디스어셈블러 한계 : 레지스터의 값을 기반으로 점프하는 간접 분기(Indirect Jump, jmp rax)의 경우, 실행 중(Runtime)이 아니면 rax에 어떤 값이 들어갈지 알 수 없으므로 정적 분석만으로는 실행 흐름을 완벽하게 추적(Control Flow Graph 구성)할 수 없다.

(예시: “**종이에 적힌 교실로 가라.**” 라고 했는데, 프로그램이 실행될 때 까지는 빈 종이 상태 즉, 실제로 실행해 봐야 rax 값과 점프 위치를 알 수 있음.)

3-1. EP(Entry Point), 디버깅 도구 활용

- EP (Entry Point, 진입점)

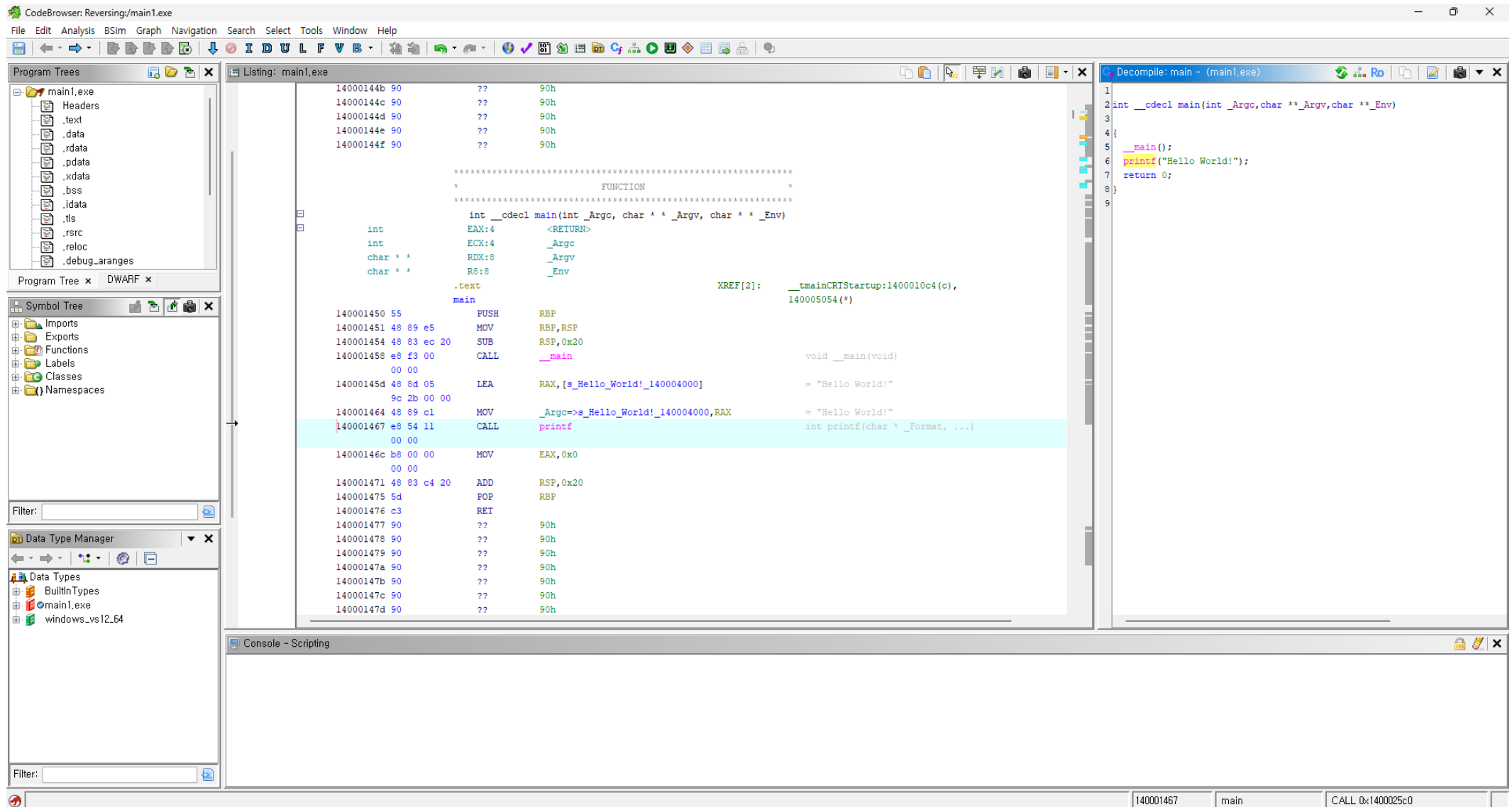
- ： 운영체제의 로더(Loader)가 바이너리 파일(PE/ELF)을 메모리에 적재한 후, 가장 처음으로 CPU의 RIP 레지스터가 가리키게 되어 실행을 시작하는 메모리 주소입니다.
- 컴파일러가 생성한 초기화 코드(CRT Startup)가 먼저 실행되며, 이후 우리가 작성한 main 함수가 호출됩니다. 리버서는 바이너리 헤더(e_entry 또는 AddressOfEntryPoint)를 분석하여 이 지점부터 분석을 시작합니다.
(실행 파일 메모리에 올리기 → Entry Point부터 시작 → 초기 준비 코드 실행 → main() 실행)



3-2. EP(Entry Point), 디버깅 도구 활용

- 도구별 주요 단축키 및 사용법
- x64dbg (동적 분석 디버거):
 - F2: 중단점(Breakpoint) 설정/해제 (INT 3 삽입)
 - F7 (Step Into): 함수 내부로 진입하여 한 줄씩 실행 (Call 내부로 들어감)
 - F8 (Step Over): 함수 내부로 진입하지 않고 한 줄씩 실행 (Call을 한 번에 실행)
 - F9: 다음 중단점까지 프로그램 계속 실행 (Run)
- IDA Pro (정적 디스어셈블러 및 디컴파일러):
 - Space: 어셈블리 뷰를 그래프 뷰(Control Flow Graph) ↔ 텍스트 뷰로 전환
 - F5: 기계어를 C언어 형태의 의사 코드(Pseudocode)로 역컴파일(Decompile)
 - X (Cross Reference): 선택한 변수나 함수가 호출되거나 참조된 모든 위치 검색
 - G: 특정 메모리 주소로 바로 이동 (Go to address)
- Ghidra (무료 오픈소스 분석 프레임워크):
 - G: 주소 이동 / L: 변수 및 함수 이름 재설정(Rename) / T: 데이터 타입 변경

4-1. Ghidra 설치



4-2. x64dbg 설치

main1.exe - PID: 29888 - 모듈: ntdll.dll - 스레드: 주 스레드 26436 - x64dbg

파일(F) 보기(V) 디버깅(D) 추적(N) 플러그인(P) 즐겨찾기(I) 설정(O) 도움말(H) May 27 2026 (TitanEngine)

CPU 로그 메모 중단점 메모리 덤프 호출 스택 SEH 스크립트 기호 소스 참조 스레드 핸들 추적

RIP: 00007FFDADABD78E

00007FFDADABD75C C64424 40 00 mov byte ptr ss:[rsp+40],0
00007FFDADABD761 48:836424 20 00 and qword ptr ss:[rsp+20],0
00007FFDADABD767 41:B9 01000000 mov r9d,1
00007FFDADABD76D 4C:8D4424 40 lea r8,qword ptr ss:[rsp+40]
00007FFDADABD772 41:8D51 10 lea edx,qword ptr ds:[r9+10]
00007FFDADABD776 48:C7C1 FFFFFFFF mov rcx,FFFFFFFFFFFFFFFF
00007FFDADABD77D E6 7E2D0400 call <ntdll.ZwQueryInformationThread>
00007FFDADABD782 85C0 test eax,edx
00007FFDADABD784 78 0A js ntdll!.7FFDADABD790
00007FFDADABD786 807C24 40 00 cmp byte ptr ss:[rsp+40],0
00007FFDADABD788 75 03 jne ntdll!.7FFDADABD790
00007FFDADABD78E EB 00 jmp ntdll!.7FFDADABD790
00007FFDADABD790 48:83C4 38 add rsp,38
00007FFDADABD794 C3 ret
00007FFDADABD795 CC int3
00007FFDADABD796 CC int3
00007FFDADABD797 CC int3
00007FFDADABD798 CC int3
00007FFDADABD799 CC int3
00007FFDADABD79A CC int3
00007FFDADABD79B CC int3
00007FFDADABD79C 48:83EC 28 sub rsp,28
00007FFDADABD79D 6548:8B0C25 60000000 mov rcx,qword ptr [0:60]
00007FFDADABD7A0 33D2 xor edx,edx
00007FFDADABD7AB 41:B8 20160100 mov r8d,11620
00007FFDADABD7B1 48:8B49 30 mov rcx,qword ptr ds:[rcx+30]
00007FFDADABD7B5 E8 DC7EFFFF call <ntdll.RtlAllocateHeap>
00007FFDADABD7BA 48:8905 7F000800 mov qword ptr ds:[7FFDADABD840],rax
00007FFDADABD7C1 48:85C0 test rax,rax
00007FFDADABD7C3 74 1A je ntdll!.7FFDADABD7E0
00007FFDADABD7C6 33D2 xor edx,edx
00007FFDADABD7C8 41:B8 20160100 mov r8d,11620
00007FFDADABD7CE 48:8BC8 mov rcx,rax
00007FFDADABD7D1 E8 5A3B0500 call <ntdll.memset>
00007FFDADABD7D6 C705 C0550800 32000000 mov dword ptr ds:[7FFDADAB72DA0],32
00007FFDADABD7E0 48:83C4 28 add rsp,28
00007FFDADABD7E3 C3 ret
00007FFDADABD7E5 CC int3
00007FFDADABD7E6 CC int3
00007FFDADABD7E7 CC int3
00007FFDADABD7E8 CC int3
00007FFDADABD7E9 CC int3
00007FFDADABD7EA CC int3
00007FFDADABD7EB CC int3
00007FFDADABD7EC 48:8BD1 mov rdx,rcx
00007FFDADABD7EF 48:85C9 test rcx,rcx
00007FFDADABD7F2 74 50 je ntdll!.7FFDADABD844
00007FFDADABD7F4 0FB749 02 movzx ecx,word ptr ds:[rcx+2]
ntdll!.00007FFDADABD790

.text:00007FFDADABD78E ntdll.dll:11D78E #11D78E

주소 Hex ASCII

00007FFDAD9A1000 CC CC CC CC CC CC CC CC CC CC CC CC CC CC CC 1111111111111111
00007FFDAD9A1010 48 89 5C 24 08 57 48 83 EC 20 4C 8B 41 38 48 88 H.\\$.WH.\\$ L.AB.
00007FFDAD9A1020 D9 48 8B FA 49 8D 48 14 65 48 89 0C 25 58 18 00 Uh.Ui.H.eh..x.
00007FFDAD9A1030 00 8B 11 F6 C2 10 75 25 48 48 48 18 48 88 D7 E8 ..0A.uh.k.H.e
00007FFDAD9A1040 3C 00 00 00 48 8B 5C 24 30 65 48 C7 04 25 58 18 <...H.\$0eh..x.
00007FFDAD9A1050 00 00 00 00 00 00 00 48 83 C4 20 5F C3 CC 45 88 80H.A..Ate..
00007FFDAD9A1060 E0 00 00 00 65 48 8B 04 25 48 00 00 44 3B 00 a...eh..h...d1A
00007FFDAD9A1070 74 C6 E8 70 8D 11 00 E8 BF CC CC CC CC CC CC CC t4e)...e1111111
00007FFDAD9A1080 48 89 5C 24 08 48 89 6C 24 18 48 89 54 24 10 56 H.\\$.H.TS.V
00007FFDAD9A1090 57 41 56 48 83 EC 20 48 8B C2 4C 8B F2 48 C1 E8 WAvH.i H.AL.dhAe
00007FFDAD9A10A0 10 41 B8 01 00 00 0F B7 D0 46 09 48 C1 E2 A.A....DH.dhAe
00007FFDAD9A10B0 06 48 03 D1 E8 93 05 00 0F B6 44 24 4C 48 C1 H.Ae....NDILHA
00007FFDAD9A10C0 E0 08 41 0F B7 F6 48 05 C0 05 00 00 C1 E6 06 48 A.A..0H.A...Ae.H
00007FFDAD9A10D0 03 C3 48 03 F3 48 3B F0 75 14 48 8B 5C 24 40 48 .AH.0H.du.H..\$0H
00007FFDAD9A10E0 8B 6C 24 50 48 83 C4 20 41 5F C3 CC 4C 48 88 .1SPH.A.A.A.A.H.
00007FFDAD9A10F0 FE BD 80 00 00 00 66 83 3F 00 74 11 0F B7 17 48 p....f.T...H
00007FFDAD9A1100 C1 E2 06 48 03 D3 66 83 7A 04 00 75 1A 48 83 C7 A.A.H.OF.z...u.H.C
00007FFDAD9A1110 02 48 83 ED 01 75 DF 45 33 C0 48 8B DE 48 8B C8 H.T.u6E3AH.OH.E
00007FFDAD9A1120 E8 C7 04 00 00 E8 82 45 23 4D 8B C6 48 8B C8 ..e12GH.EH.E
00007FFDAD9A1130 F8 17 24 04 00 E8 D6 C7 CC CC CC CC CC CC CC ..A..e111111111111111

명령: 명령어는 실패로 구분되어 있습니다.(여분불러 명령어 처리): mov eax, ebx

원치 중지됨 업데이트 확인을 위해 Github에 연결

RAX 0000000000000000
RBX 00007FFDAD83A090 "LdrpInitializeProcess"
RCX 00007FFDAD800514 ntdll!.00007FFDAD800514
RDX 0000000000000000
RBP 0000000000000000
RSP 000000C8325FF540 &"LdrpInitializeProcess"
RSI 00007FFDAD817700 "minkernel\\ldr\\ldrinit.c"
RDI 000000C83228D000

R8 000000C8325FF538
R9 0000000000000000
R10 0000000000000000
R11 0000000000000246 L'Z'
R12 0000000000000001
R13 0000000000000000
R14 0000000000000040 '@'
R15 000001F966220000

RIP 00007FFDADABD78E ntdll!.00007FFDADABD78E

EFLAGS 0000000000000246
ZF 1 PF 1 AF 0
OF 0 SF 0 DF 0
CF 0 TF 0 IF 1

LastError 00000000 (ERROR_SUCCESS)
LastStatus 00000000 (STATUS_SUCCESS)

GS 002B FS 0053
ES 002B DS 002B
CS 0033 SS 002B

ST(0) 000000000000000000000000 x87r0 비어 있음 0.00000000000000000000
ST(1) 000000000000000000000000 x87r1 비어 있음 0.00000000000000000000
ST(2) 000000000000000000000000 x87r2 비어 있음 0.00000000000000000000
ST(3) 000000000000000000000000 x87r3 비어 있음 0.00000000000000000000
ST(4) 000000000000000000000000 x87r4 비어 있음 0.00000000000000000000
ST(5) 000000000000000000000000 x87r5 비어 있음 0.00000000000000000000
ST(6) 000000000000000000000000 x87r6 비어 있음 0.00000000000000000000
ST(7) 000000000000000000000000 x87r7 비어 있음 0.00000000000000000000

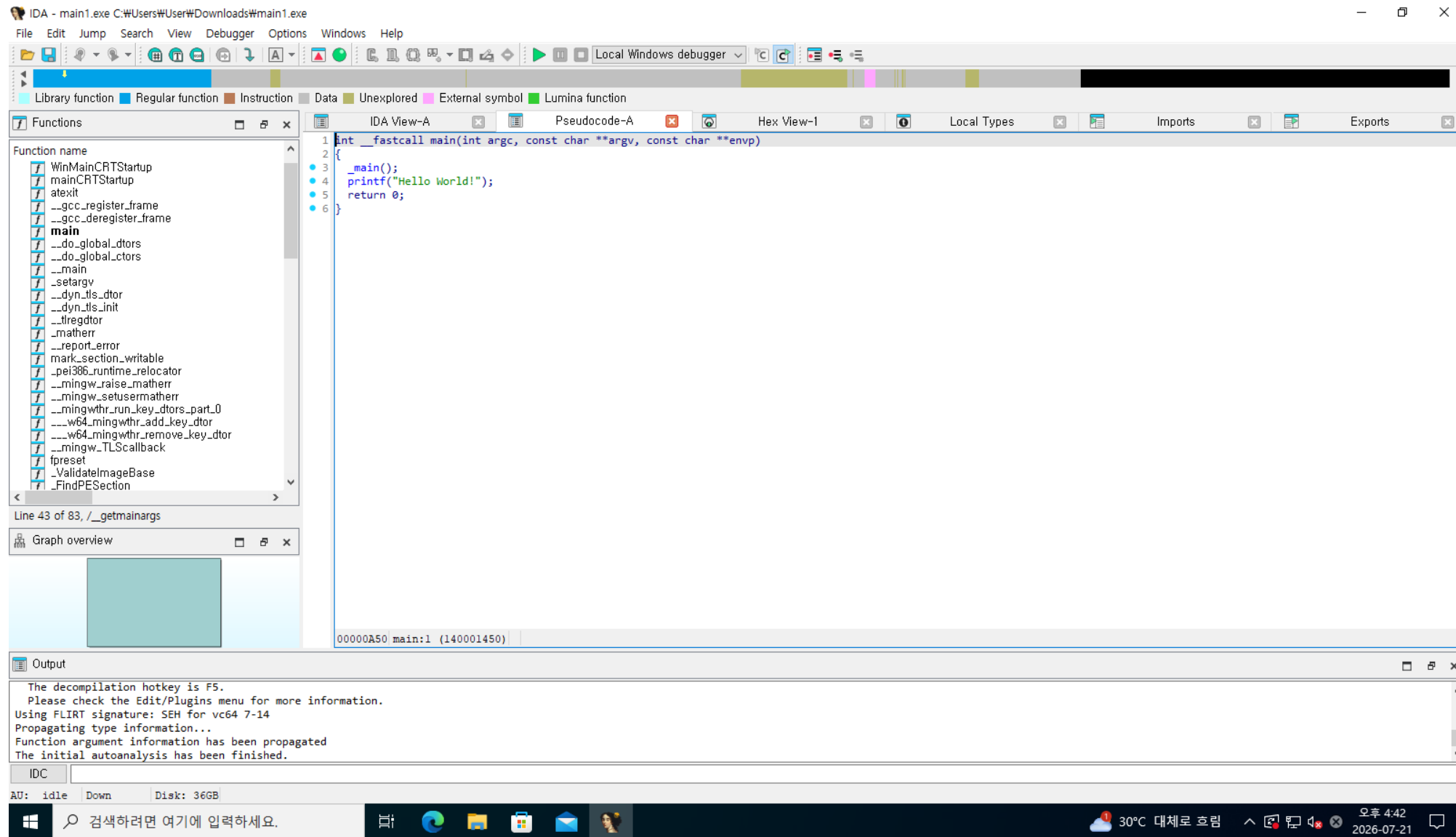
기본값 (x64 fastcall) 5
1: rcx 00007FFDAD800514 ntdll!.00007FFDAD800514
2: rdx 0000000000000000 0000000000000000
3: r8 000000C8325FF538 000000C8325FF538
4: r9 0000000000000000 0000000000000000
5: [rsp+28] 0000000000000040 0000000000000040

000000C8325FF540 00007FFDAD83A090 ntdll!_flttused+F908
000000C8325FF548 00007FFDAD97191 ntdll!.RtlQueryElevationFlags+241로 빈칸 (출발: ntdll.RtlAllocateHeap)
000000C8325FF550 0000000000000000
000000C8325FF558 000000C8325FF5B0
000000C8325FF560 0000000000000000
000000C8325FF568 0000000000000040
000000C8325FF570 000000C83228D000
000000C8325FF578 00007FFDAD2083A ntdll!.EtwEventWriteNoRegistration+254A로 빈칸 (출발: ntdll.RtlGetReturnA
000000C8325FF580 00007FFDAD83A000 ntdll!_flttused+F878
000000C8325FF588 00007FFDAD817700 ntdll!.RtlNtdllName+3950
000000C8325FF590 00007FFDAD817700 ntdll!.RtlNtdllName+3950
000000C8325FF598 000000C83228D000
000000C8325FF5A0 00007FFDAD83A570 ntdll!_flttused+FD8
000000C8325FF5A8 000001F9662E5668
000000C8325FF5B0 000001F9662E568
000000C8325FF5B8 000000C8325FF640
000000C8325FF5C0 00007FFDAD8720B0 ntdll!.RtlpFreezeTimeBias+49E0
000000C8325FF5C8 0000000000000000
000000C8325FF5D0 0000000000000000
000000C8325FF5D8 00000001325FFA00
000000C8325FF5E0 0000000000000000
000000C8325FF5E8 00000000000000620062

기본값

디버깅에 사용한 시간: 0:00:00:25

4-3. IDA 설치



5. kknock.asm 파일 분석 및 복원

CLang > ASM main.asm

```
1  ; int sub_140001000(unsigned int a, unsigned int b);
2  ; 인자 매핑 : ecx = a, edx = b
3
4  sub_140001000:
5      test    ecx, 1          ; ecx(a)의 가장 오른쪽 비트(LSB)가 1인지 AND 연산으로 테스트 (홀수/짝수 판별)
6      jnz     short loc_140001025 ; 결과가 0이 아니면(홀수이면), loc_140001025로 점프 (에러 반환 지점)
7
8      mov     eax, ecx        ; eax = a
9      mov     r8d, edx        ; r8d = b
10
11     test    r8d, r8d        ; r8d(b)가 0인지 테스트
12     je      short loc_140001024 ; b가 0이면, loc_140001024(종료 및 eax 반환)로 점프
13
14 loc_140001010:
15     xor     edx, edx        ; edx를 0으로 초기화. (div 연산을 위해 edx:eax를 세팅)
16     div     r8d             ; edx:eax 조합을 r8d(b)로 나눔
17     ;       |       |       |       |       ; 몫은 eax에, 나머지는 edx에 저장 (edx = a % b)
18
19     mov     eax, r8d        ; eax = b (이전 나눔수)
20     mov     r8d, edx        ; r8d = edx (나머지)
21
22     test    r8d, r8d        ; 나머지(r8d)가 0인지 테스트
23     jne     short loc_140001010 ; 나머지가 0이 아니면 루프(loc_140001010)로 돌아감
24
25 loc_140001024:
26     ret                     ; eax에 저장된 값을 반환하고 함수 종료
27
28 loc_140001025:
29     mov     eax, -1         ; eax = -1
30     ret                     ; -1 반환
```

CLang > C main.c > sub_140001000(unsigned int, unsigned int)

```
1  int sub_140001000(unsigned int a, unsigned int b) {
2      // 첫 번째 인자 a가 홀수인지 검사 (test ecx, 1)
3      if (a & 1) {
4          return -1; // 홀수이면 -1 반환 (loc_140001025)
5      }
6
7      // b가 0인지 확인. 0이면 루프를 타지 않고 a를 바로 반환 (test r8d, r8d)
8      if (b == 0) {
9          return a;
10     }
11
12     // 루프 로직
13     unsigned int r;
14     while (b != 0) {
15         r = a % b; // xor edx, edx 후 div r8d 연산의 결과 (나머지는 edx)
16         a = b;           // mov eax, r8d
17         b = r;           // mov r8d, edx
18     }
19
20     return a; // 루프 종료 후 a (최대공약수) 반환
21 }
```



감사합니다

17기 최동현